Lab 1: Intepreting Expressions

Goal: Set up the lab environment and implement a visitor-based interpreter for MiniJava expressions, defined by ANTLR4 grammar files. The interpreter must evaluate a single expression read from a file and print the result to standard output.

Deadline: 12:00:00 Apr 6 (Hard Deadline, No extension)

Programming Language: Java Only.

Task 1: Set Up the Environment

1.1 Installing JDK 21+

Before the experiment, we need to set up the Java environment. If you do so manually, download and install a popular Java IDE, e.g., IntelliJ IDEA.

MacOS

```
$ brew install openjdk@21
```

Add `export JAVA_HOME=$(/usr/libexec/java_home -v 21)` at the end of the file `~/.zshrc` (or`~/.bashrc`).

Ubuntu / Debian

```
$ sudo apt update && sudo apt install openjdk-21-jdk
```

Windows

You may refer to the Tutorial to download and install OpenJDK 21 (or upper). Please make sure the environment variable `JAVA_HOME` is set and the command `java` is on your `PATH`.

Check after Installation

```
$ java -version # Expected: openjdk version "21.x.x" (or later)
```

1.2 Installing ANTLR4

```
$ pip install antlr4-tools
```

After installing JDK and ANTLR, you may also want to download the ANTLR plugin in IntelliJ IDEA for convenience.

Task 2: Understand the Grammar

Open the maven project (that we provided with the lab document) in IntelliJ IDEA or any IDE you prefer. The project is organized as follows:

```
lab1-expr/
├── pom.xml                                # Maven build config
├── src/main/java/cn/edu/nju/cs/
│   ├── Main.java                          # Entry point
│   ├── MiniJavaLexer.g4                   # Grammar for lexer
│   ├── MiniJavaParser.g4                  # Grammar for parser
│   │  # following files are generated by ANTLR4.
│   │  # we don't recommend you to modify them manually
│   ├── MiniJavaLexer.interp               # Meta data for lexer
│   ├── MiniJavaParser.interp              # Meta data for parser
│   ├── MiniJavaLexer.tokens               # Token types for lexer
│   ├── MiniJavaParser.tokens              # Token types for parser
│   ├── MiniJavaLexer.java                 # The lexer
│   ├── MiniJavaParser.java                # The parser
│   ├── MiniJavaParserVisitor.java         # The visitor interface
│   ├── MiniJavaParserListener.java        # The listener interface
│   ├── MiniJavaParserBaseListener.java    # The base implementation
│   └── MiniJavaParserBaseVisitor.java     # The base implementation
└── testcases/
    ├── input1.txt
    ├── input2.txt
    └── ...                                # Test cases for your
interpreter
```

Read the grammar files `MiniJavaLexer.g4` and `MiniJavaParser.g4`. The former defines the regular expression for lexical analysis and the latter defines the grammar of expressions for syntax analysis. Particularly, read `MiniJavaParser.g4` carefully. It defines all possible expressions the interpreter needs to deal with.

2.1 Types and Literals

In this lab, the interpreter must support literals of four possible types:

Type	Examples	Notes
`int`	`42`	32-bit signed two's complement (Java `int`)
`char`	`'A'`	8-bit signed integer (`-128..127`). We do not have escaped char, e.g. `'\n'`,`'\"'`, etc.
`boolean`	`true`, `false`	
`string`	`"hello"`, `""`	We don't have escaped char, e.g., `'\n'`, in a string. A string can be empty, i.e., `""`.

2.2 Operator Precedence

Listed precedence from **tightest** (highest precedence, top) to **loosest** (bottom):

Level	Operator(s)	Associativity	Operand Types	Result Type
1	`not`, `~`, `+`, `-`, `++`, `--`, cast	Prefix/Postfix	see below	see below
2	`*`, `/`, `%`	Left	integral × integral	`int`
3	`+`, `-`	Left	integral × integral; string concatenation for `+`	`int` / `string`
4	`<<`, `>>`, `>>>`	Left	integral × integral	`int`
5	`<`, `>`, `<=`, `>=`	Left	integral × integral	`boolean`
6	`==`, `!=`	Left	compatible types	`boolean`
7	`&`	Left	integral × integral	`int`
8	`^`	Left	integral × integral	`int`
9	`|`	Left	integral × integral	`int`
10	`and`	Left	boolean × boolean	`boolean`
11	`or`	Left	boolean × boolean	`boolean`
12	`? :`	Right	boolean × any × any	any

Note 1: The operator `( )` would be the highest precedence than others.

Note 2: `integral` means `int` or `char` in the table above.

Note 3: Following Java's computation rule, before a (unary/binary) arithmetic and bitwise operators, `char` is promoted to `int` by a signed extension before computation. Thus, the computation result is of the type `int`.

Task 3: Visitor of the Syntax Tree

Let us generate a visitor of the syntax tree via ANTLR4:

```
$ cd src/main/java/cn/edu/nju/cs
$ antlr4 -visitor -package cn.edu.nju.cs MiniJavaLexer.g4
$ antlr4 -visitor -package cn.edu.nju.cs MiniJavaParser.g4
```

The command above will re-generate the `.java` files in the `src/main/java/cn/edu/nju/cs` directory. **Note that this is a dangerous operation because it will overwrite all generated `.java` files in this directory.**

Task 4: Implementing the Interpreter

Below are the detailed semantics with expression examples. All examples can be used as your preliminary test cases.

4.1 Unary Operation

Unary + / −

The operand should be either `int` or `char`.

```
+1        → 1
−1        → −1
+'a'      → 97
−'a'      → −97
−false    → Error
```

Logical NOT

The operand must be of `boolean`; otherwise，return an error.

```
not true   → false
not false  → true
not 42     → Error
```

Bitwise NOT

Operand must be integral (`int` or `char`); result is `int`.

```
~0     → −1
~5     → −6
~true  → Error
```

Type Cast

Java-style cast is supported for integral types:

Form	Operand	Result
`(int) expr`	`int`, `char`	`int`
`(char) expr`	`int`, `char`	`char` (signed 8-bit truncation)

Rules:

- `(int) charValue` converts `char` to `int` using sign extension.
- `(char) intValue` converts using signed 8-bit truncation semantics.

```
(char) 56      → '8'
(int) 'c'      → 99
(char) 300     → ','
```

Increment / Decrement

```
++30     → Error
30--     → Error
```

4.2 Multiplication/Division/Modulo

Left Oprand	Right Oprand	Result
int	int	`int` (integer arithmetic)
int	char	`int` (char promoted to int)
char	int	`int` (char promoted to int)
char	char	`int` (both promoted to int)

Note 1: Integer overflow wraps silently: `2147483647 * 2` = `-2`.

Note 2: Any other combination of operand types yields an error.

Note 3: Division by zero yields an error.

4.3 Addition/Subtraction

The operator `+` supports integral addition and string concatenation. If either operand is `string`, `+` performs string concatenation. The operator `−` does not support string operands.

Left Oprand	Right Oprand	Result
int	int	`int`
int	char	`int` (char promoted to int before computation)
int	string	`string` (for `+` only)
char	int	`int` (char promoted to int before computation)
char	char	`int` (both promoted to int before computation)
char	string	`string` (for `+` only)
boolean	string	`string` (for `+` only)
string	string	`string` (for `+` only)
string	int	`string` (for `+` only)
string	char	`string` (for `+` only)
string	boolean	`string` (for `+` only)

Note: Any other combination of operand types yields an error.

```
"foo" + "bar"    → "foobar"
"foo" + 42       → "foo42"
1 + "a"          → "1a"
"foo" - "bar"    → Error
'A' + 1          → 66
'A' + "!"        → "A!"
"A" + true       → "Atrue"
```

4.4 Shifting Operation

Both operands must be integral (`int` or `char`).

Operator	Description	Example
`<<`	Signed left shift	`1 << 4` = `16`
`>>`	Signed right shift	`-1 >> 1` = `-1`
`>>>`	Unsigned right shift	`-1 >>> 1` = `2147483647`

```
1 << 31    → -2147483648   (sign bit set)
'A' << 1   → 130
true << 2  → Error         (shifts are integral-only)
```

4.5 Relational Operation Except for == and !=

Only integral types (`int`, `char`) support relational operators. **Mismatched incompatible types produce Error**.

```
3 < 4          → true
'A' < 100      → true
true < 100     → Error
```

4.6 Equality (==, !=)

Left	Right	Behavior
int	int	compare the values
int	char	Promote char→int, compare
char	int	Promote char→int, compare
char	char	compare the values
boolean	boolean	compare the values
string	string	compare the strings

Note: Any other combination of operand types yields an error.

```
'A' == 65       → true
1 == true       → Error
```

4.7 Bitwise Operation

Operands must be integral (`int` or `char`) and are evaluated as `int`.

Operator	Example	Result
`&`	`255 & 15`	`15`
`^`	`12 ^ 10`	`6`
`|`	`10 | 5`	`15`
`~`	`~0`	`-1`

4.8 Logical Operation

Both operands must be of `boolean` and short-circuit evaluation is required:

- `and`: if left side is `false`, right side is not evaluated.
- `or`: if left side is `true`, right side is not evaluated.

```
true and false        → false
1 and true            → Error
```

4.9 Ternary

`cond ? thenExpr : elseExpr`

- `cond` must be of `boolean`; otherwise Error.
- **Right-associative:** `a ? b : c ? d : e` → `a ? b : (c ? d : e)`.

```
true  ? 1 : 2             → 1
```

4.10 Input and Output Format

Your interpreter will be executed with a file containing a single expression as input.

e.g., the file `input.txt` contains:

```
1 + 1 * 3
```

your interpreter will be executed as:

```
$ java -jar your_interpreter.jar input.txt
```

We will expect your output to be(we don't care about the breakline at the end of the output):

```
4
```

if the input expression is invalid or can't be evaluated successfully. we will expect an error message `Process exits with 34.` and the process exits with code `34` (you can use `System.exit(34)`).

The interpreter prints the **result** of the expression:

Value type	Display format	Example
`int`	Decimal integer	`42`, `-1`
`char`	Single-quoted character	`A`
`boolean`	`true` / `false`	`true`
`string`	A string without quotation marks	`hello`
Error	A string with the error code	`Process exits with 34.`

Case 1: Normal Output

```
"foo" + "bar"    → foobar
```

Case 2: Error Output

```
1 + true    → Process exits with 34.
```

Case 3: Empty String Output

```
""    → (prints nothing)
```

Case 4: Combination of Multiple expressions

```
1 > 0 ? (2 > 3 ? "a" : (4 > 5 ? "b" : "c")) : "d" → c
```

Case 5: Invalid Expression

```
1 + * 2    → Process exits with 34.
```

Task 5: Submission

Submit your source files as a zip to our OJ system. OJ will use the maven files to build the interpreter. OJ then tests the correctness of the output. We do not consider the efficiency of the interpreter in this lab.

Note: The `.zip` should follow the structure below:

```
submission.zip
└── outer_folder/
    ├── src/
    └── pom.xml
```